Context Engineering 前置知识

> 这份文档为零基础读者准备。如果你已经熟悉 System Prompt、Prompt Caching、上下文窗口、KV Cache（Key-Value Cache，键值缓存）等概念，可以直接翻阅主报告。

一、Context Engineering 到底是什么？

Context Engineering = 设计模型"能看到什么"的系统工程。

你看过一个让 AI 写代码的场景吗？用户告诉模型"帮我重构这个函数"，模型不仅看到这句话，还看到了整个代码库的上下文——当前文件、相关接口、历史修改记录。

这背后就是 Context Engineering 在做的事：**决定什么信息放进模型的上下文窗口、以什么格式、放多少、什么时候更新。**

核心矛盾：模型的上下文窗口是有限的（虽然已经从 4K 长到 200K-1M），但一次 AI 调用可能需要的信息量远超这个限制。Context Engineering 就是解决这个"塞不下的问题"。

二、核心概念

Token（词元）

Token 是大模型处理文本的最小单位。简单说：**模型不读"字"，它读"词元"。**

- "你好世界" = 4 个 token（你/好/世/界）
- "Hello world" = 2 个 token（Hello / world）
- 一篇 1000 字的中文文章 ≈ 800-1500 个 token

在 Context Engineering 中，token 是核心度量单位——上下文窗口能装多少 token、每次调用花多少钱（按 token 计费）、缓存命中省了多少 token。**理解 token 才能理解这个领域的工程权衡。**

上下文窗口（Context Window）

模型一次能"看到"的最大 token 数。类比工作记忆：

年份	典型大小	类比
2022	4K tokens	一页 A4 纸
2023	32K–100K	一本短篇小说
2024	128K–200K	一本长篇小说
2025–2026	200K–2M	三本《三体》的厚度

但长 ≠ 好。"Lost in the Middle" 现象：信息放在 prompt 的开头或结尾时模型能准确提取，放中间就很容易被忽略。所以**怎么放跟放了多少同样重要**。

System Prompt（系统提示）

模型的"岗位说明书"——告诉模型它是什么角色、有什么工具、怎么回答问题。2023 年只要 "You are a helpful assistant"（几十个 token），2026 年的 Agent system prompt 经常长达数千甚至数万 token，包含：

- 角色定义
- 行为规则和边界
- 所有工具的描述和参数 schema
- 输出格式要求

Prompt Caching（提示缓存）

如果每次请求都有大量相同的"静态内容"（如 system prompt、工具定义），模型不得不重新处理这些相同的 token——浪费算力和时间。

Prompt Caching 的解决方案：把计算过的 prompt 状态缓存起来。下次请求的前缀跟缓存内容匹配时，直接复用计算结果。

效果：延迟降低 80%+，输入 token 成本降低 90%。

定价机制（理解 cache 经济学的关键）：

概念	含义	定价影响
Cache Write	第一次写入缓存（处理原始 prompt）	比普通输入贵 25–100%（各厂商不等）
Cache Read	命中缓存（复用计算结果）	约为普通输入价格的 10%
TTL	缓存过期时间（各厂商 5min–24h 不等）	过期后需要重新 Cache Write

核心 trade-off：Cache Write 更贵，Cache Read 更便宜。适合"写一次、读很多次"的场景。如果你每次的 system prompt 都不一样，caching 不仅不省钱还更贵。

场景	是否适合 Caching
每次相同的 system prompt	✅ 极适合
每次相同的一组工具定义	✅ 极适合
每次不同的用户问题	❌ 不适合
每次不同的 RAG 结果	❌ 不适合

KV Cache（键值缓存）

Transformer 模型在生成每个 token 时，需要重新计算之前所有 token 的注意力。KV Cache 把之前计算好的结果存下来，避免重复计算。

这是推理加速的核心技术。Context Engineering 中的压缩技术很多都在 KV Cache 层面做文章——不是压缩"输入文本"，而是压缩"模型内部的中间计算结果"。

RAG（检索增强生成）

RAG（Retrieval-Augmented Generation）是让模型在回答问题前先去外部知识库"查资料"的技术。流程极简版：

```
收到问题 → 从知识库检索相关文档 → 把文档拼进 prompt → 模型基于资料回答
```

RAG 是 Context Engineering 最经典的应用场景——**把"外部知识"动态注入上下文窗口**。难点在于查什么、查多少、结果怎么放、放什么位置——这些全都是 context 编排的问题。

MCP（Model Context Protocol）

Anthropic 在 2024 年底推出的开放协议，标准化了 Agent 与外部工具的连接方式，被称为"AI 的 USB-C"。到 2026 年已被 ThoughtWorks 评为最具影响力的 AI 基础设施之一。

MCP 对 Context Engineering 的意义：它统一了"工具定义和数据源怎么注入上下文"的格式。之前每个框架各搞一套，MCP 之后工具提供方只需实现一个 Server，所有兼容 Agent 都能拿到格式统一的上下文。

Attention Budget（注意力预算）与 Context Rot（上下文衰减）

Anthropic 在 2025 年提出的两个重要概念：

Attention Budget：Transformer 的注意力机制决定了 token 越多，每个 token 分到的注意力越稀疏。就像一张照片——像素越多，每个像素的细节越少。所以上下文窗口不能无限填。

Context Rot：随着上下文增长，模型准确回忆早期信息的能力会渐进式下降。不是突然忘记，而是越来越模糊——类似人记不住半年前的聊天细节。这对 Agent 系统的影响是：长对话中模型越往后越"迷失"在前面的信息里。

Memory（记忆）

多轮对话中，模型需要记住之前聊过什么。Context Engineering 中的"记忆"不是一个东西，而是一套策略：

策略	做法	适用
滑动窗口	只保留最近几轮对话	简单闲聊
摘要压缩	把旧对话总结成几句话	长对话
向量检索	把历史存入向量库，按需检索	超长历史

上下文压缩——把 prompt "挤一挤"

除了管理记忆，Context Engineering 还直接压缩 prompt 本身。主要方法：

LLMLingua（Microsoft, 2023）：用一个小模型识别出 prompt 中"不太重要"的 token 删掉，实现 2–20x 压缩。效果最好的场景是 RAG——挤掉文档中的噪音后，模型反而表现更好。这是唯一大规模工程落地的压缩技术。

StreamingLLM（MIT & Meta, 2023）：发现了 Attention Sink 现象——prompt 的第一个 token 即使语义无关也会获得极高的注意力分数。利用这个特性，模型可以处理无限长度序列——只保留初始 token + 最近窗口，其他全部丢掉。

RadixAttention（SGLang, 2024）：在推理引擎层面，用基数树自动复用 KV Cache。common prompt 前缀（如工具定义、few-shot 示例）会被自动共享——多个请求不需要重复计算相同的开头部分，吞吐量最高提升 5x。这是"让引擎自己管理上下文"的路线，开发者不需要手动优化。

三、为什么 2025 年突然火了？

三个趋势在 2025 年撞在一起：

1. **Agent 系统爆发**：AI 不再只是"问一句答一句"，而是自主调用工具、多步推理。一次 Agent 调用的上下文包含 system prompt + 工具定义 + 历史对话 + RAG 结果 + 当前问题——怎么编排这些信息成了一个系统工程问题。
1. **模型上下文窗口变大**：2024–2025 年，Gemini 1.5 率先达到 1M token，其他模型跟进。但社区发现：塞满窗口并不会让模型表现更好——**关键不是能放多少，而是怎么放**。
1. **成本经济变化**：Prompt Caching 的出现彻底改变了"prompt 多长算贵"——静态内容可以缓存，边际成本趋近于零。

2025 年 6 月，Shopify CEO Tobi Lutke 在 X 上发了条推文正式命名了这个领域。Andrej Karpathy 用一个比喻让它迅速传播：

> LLM = CPU（处理器）

> Context Window = RAM（工作内存）

> Context Engineering = 操作系统（决定什么数据加载到内存）

四、Context Engineering vs Prompt Engineering

	Prompt Engineering	Context Engineering
核心问题	"怎么写指令"	"给模型看什么"
范围	prompt 文本	全部上下文（system + tools + history + RAG）
关注点	措辞、格式、推理策略	信息架构、token 预算、缓存、压缩
静态 vs 动态	相对静态	每次推理动态组装
可以类比	写一份好文档	设计一个信息管理系统

关系：Prompt Engineering 是 Context Engineering 的子集。你还在写 prompt，但现在的"prompt"是一个包含大量动态组件的复杂系统。

五、一张图总结

```
                    Context Engineering
                            │
        ┌───────────────────┼───────────────────┐
        │                   │                   │
    工程层面            工具层面            策略层面
        │                   │                   │
      ┌─┴─┐             ┌─┴─┐             ┌─┴─┐
      │   │             │   │             │   │
  System  Token     Prompt  Memory    RAG   Context
  Prompt  预算     Caching 管理     检索    压缩
      │   │             │   │             │   │
    角色  放多少      省成本  记住历史   找外部  挤空间
    定义  才够?
```